Programming with Generative AI

Viraj Kumar
Kotak-IISc AI-ML Centre

Lecture 10:

- Iteration patterns
- List operations



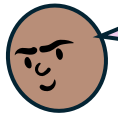
A CSR Initiative by



NPTEL

What is this function's purpose?

```
1  def mystery(data: list) -> bool:
2      if data == []:
3          return False
4      if data[0] % 2:
5          return True
6      return mystery(data[1:])
7
8  print(mystery([0.0, -1, '0']))
```



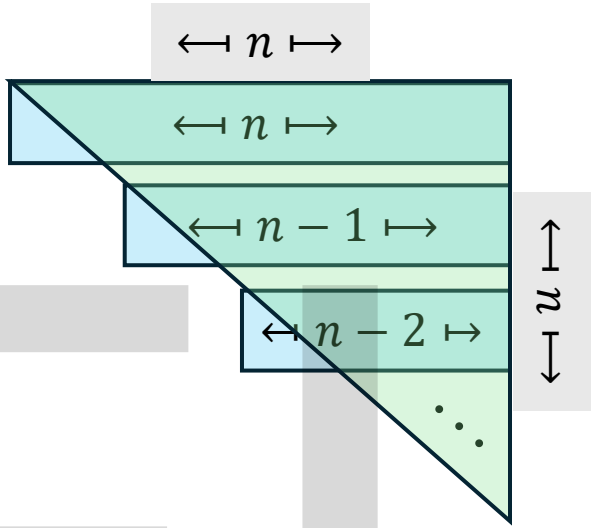
It checks if any item in data is ~~even~~ odd

PythonTutor

Heterogenous list

A better solution

- More **robust**
- More **efficient worst-case**
  - Linear vs. Quadratic



`['0', 0.0, -1]`

PythonTutor

- Examine each item in `data` **iteratively**
  - If this item is an odd int, `return True`
  - Otherwise, continue to the next item
- If we find no odd int item: `return False`

```
1  def has_odd(data: list) -> bool:
2      for item in data:
3          if type(item) == int and item % 2:
4              return True
5      return False
```

NPTEL

Pattern 1: Iterate until “success”

```
2       for item in data:
3           if type(item) == int and item % 2:
4               return True
5       return False
```

A common error:

```
for item in sequence:
    if item has property:
        success  # return/...
    else:
        failure
```

NPTEL

Examples of Pattern 1

```
def first_vowel(s: str) -> str:
    """Return the first vowel in s.
    >>> first_vowel('hello')
    'e'
    >>> first_vowel('sky')
    ''
    >>> first_vowel('Act')
    'A'
    """
    for c in s:
        if c.lower() in 'aeiou':
            return c
    return ''
```

Modern type-hinting syntax

```
-> int:
t in data.


)
```



What about 'über'?

NPTEL

Pattern 2: Accumulate

```
def prime_product(data: list[int]) -> int:
    """Return the product of the prime numbers in data.

    In general:

    accumulator = initial
    for item in sequence:
        update accumulator
    """
    product = 1
    for i in data:
        if is_prime(i):
            product *= i
    return product
```

NPTEL

Is this Pattern 1 or Pattern 2?

```python
def is_prime(n: int) -> bool:
    """Return True if n is prime, False otherwise.
    >>> is_prime(2)
    True
    >>> is_prime(3)
    True
    >>> is_prime(4)
    False
    """
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

PythonTutor

NPTEL

Is this Pattern 1 or Pattern 2?

```
def prime_product(data: list[int]) -> int:
    product = 1
    for i in data:
        if i >= 2:
            for j in range(2, int(i**0.5) + 1):
                if i % j == 0:
                    break  # exit this (inner) loop
            else:  # nobreak
                product *= i
    return product
```

Pattern 2

Pattern 1

PythonTutor

Examples of Pattern 2

```
def primes(data: list[int]) -> list[int]:
    """Return a list of all prime numbers in data.
    >>> primes([2, 3, 4])
    [2, 3]
    >>> primes([4, 6, 8])
    []
    """
    return [i for i in data if is_prime(i)]
```

```
    for i in data:
        if is_prime(i):
            result.append(i)
    return result
```

```
        if c.lower() in 'aeiou':
            count += 1
    return count
```

List comprehension

Similar to set-builder notation in M

$$\{i \in \text{data} : \text{is_prime}(i)\}$$



Instead, how about:
`result = result + [i]`

PythonTutor

Summary

- We often **iterate** over items in a sequence:
  - `list`, `str`, `tuple`, …
- Iteration often more **efficient** than recursion
  - *Special form*: **comprehension**
  - But recursion is a powerful idea! (“*self-help*”)
- Common iterative patterns:
  - **Pattern 1**: Iterate until success (`break`…`else`)
  - **Pattern 2**: Accumulate (`list.append` is fast!)
- Patterns can be combined (e.g., nested)
- Gen AI can help with writing **tests**

NPTEL